King Salman International University

Faculty of Computer Science and Engineering

Project Title

Intelligent Robot for Flame Location Detection

A Thesis Submitted to Computer Engineering Department
Faculty of Computer Science & Engineering, KSIU

In Partial Fulfillment of **B.Sc** Degree in Computer Engineering
Computer Engineering Program

By

Bassel Ashraf Elbahnasy

Supervisors

Dr. Mahmoud Zaki

Assistant Professor, Faculty of Computer Science & Engineering

King Salman International University

EGYPT 2026

Abstract

This report details the hardware architecture, electrical circuit design, and software integration of an Intelligent Mobile Robot designed for autonomous flame location detection and suppression. Built on a 4-wheel drive skid-steer chassis, the physical platform is driven by high-torque JGB37-545 DC geared motors controlled via dual BTS7960 motor drivers, and is powered by a robust 22.2V Li-Po dual-battery system. The core onboard computational engine is a Raspberry Pi 4 Model B, which handles local navigation, sensor data acquisition, and actuator control.

To optimize power efficiency and maintain real-time responsiveness, the system employs a distributed split-node architecture. A stationary external unit acts as the global vision node, utilizing a TP-Link Tapo C210 camera and dual custom Convolutional Neural Networks (CNNs) to identify fire sources and prioritize the detection of humans in danger.

The mobile edge node (Raspberry Pi 4 Model B) uses an Okdo LiDAR HAT for 360-degree obstacle detection and a Raspberry Pi Camera Module V3 for local perception. By computing real-time extrinsic transformation matrices using OpenCV and ArUco fiducial markers, the robot maps the 2D visual data into a 3D physical workspace. This allows the ROS 2 Navigation Stack (Nav2) to dynamically generate collision-free paths to the fire source. Upon successful arrival, the system autonomously triggers a 24V water pump via an opto-isolated relay to extinguish the flame. This document provides a comprehensive specification of the selected components, kinematic models, power distribution circuitry, and the ongoing development roadmap toward full autonomous operation. The system's autonomous navigation and collision avoidance logic were successfully validated through high-fidelity Gazebo Harmonic simulations, utilizing custom occupancy grids and tuned friction parameters to ensure reliable skid-steer performance.

Table of Contents

1. Chapter 1: Introduction

1.1. Project Background and Motivation

Fire hazards in industrial environments, multi-level warehouses, and petrochemical facilities present catastrophic risks to human life, physical inventory, and structural integrity. Traditional stationary fire protection frameworks—such as localized smoke alarms and ceiling-mounted sprinkler networks—frequently suffer from spatial limitations, latency in activation, and severe collateral water damage to high-value assets. To mitigate these deficiencies, autonomous mobile robotics has emerged as a crucial paradigm shift, removing human personnel from highly toxic, structurally compromised containment zones while deploying targeted, real-time fire suppression vectors.

Project "**Phoenix**" at King Salman International University (KSIU), addresses this critical gap through an Intelligent Mobile Robot designed for autonomous flame location detection, life-safety asset prioritization, and local fire extinguishing.

1.2. System Architecture Overview

The overarching systemic framework utilizes a distributed, split-node edge computing topography. A stationary global vision station (laptop node) monitors the environment via a wide-area surveillance camera, executing twin Convolutional Neural Networks (CNNs) to localize fire boundaries and map human targets actively in danger. Real-time coordinates are packaged into a JSON payload and streamed over a low-latency local Mosquitto MQTT broker to the mobile edge computing node. This specialized, asynchronous pipeline ensures that high-intensity neural computing is safely decoupled from the physical platform, reducing mobile thermal loading and battery consumption while ensuring rapid navigational path replanning.

1.3 Scope of the Author's Thesis Work

While Project Phoenix is an interdisciplinary team effort, this thesis explicitly focuses on the architectural engineering, physical construction, and algorithmic integration managed strictly by the author. The core scope of this independent research comprises:

- **Physical Drivetrain Architecture:** Engineering and assembling a robust, 4-wheel drive skid-steer mobile chassis driven by high-torque JGB37-545 DC geared locomotion motors.

- **Split-Rail Power Infrastructure:** Designing, isolating, and validating a mixed-voltage distribution network powered by a 22.2V Li-Po battery system. This includes implementing XL4015 buck regulation circuitry to isolate high-noise inductive servo and actuator loads from sensitive digital control elements.
- **Low-Level Embedded Electronics Integration:** Direct GPIO mapping and logic-level protection schemes interfacing the central Raspberry Pi 4 Model B microprocessing core with dual BTS7960 high-current H-bridge drivers and opto-isolated suppression relays.
- **Kinematic and Transformation Modeling:** Formalizing the spatial and velocity models governing the skid-steer chassis, including inverse/forward kinematics embedded with empirical slip factors c to account for lateral turning friction. This encompasses the initialization of internal homogeneous transformation matrices T resolving the coordinate offsets linking the robot's base link to its physical perception and actuation layers.
- **ROS 2 Low-Level Control Package Optimization:** Developing and debugging native hardware-interface script architectures—namely motor_controller.py and pump_controller.py—to seamlessly bridge high-level ROS 2 Nav2 geometry velocity vectors (/cmd_vel) and asynchronous goal states into physical hardware manipulation.
- **High-Fidelity Simulation Validation:** Creating a custom Unified Robot Description Format (URDF) blueprint and deploying it within a physics-enabled Gazebo Harmonic simulation environment. This simulation verified collision boundaries, tuned skid-steer asymmetric friction parameters (μ_1, μ_2), and validated autonomous path planning over virtual occupancy grids prior to physical hardware deployment.

2. Chapter 2: Electrical and Power Systems Design

This section describes the electrical circuit design implemented in the project, focusing on power distribution, voltage regulation, and component interfacing. The system is engineered to deliver stable and efficient power to multiple subsystems using dual-battery architecture and step-down converters.

2.1. Power Source Configuration

- **Battery Setup:** Two **11.1 V 5200 mAh Li-Po batteries** are connected in **series**, providing a total supply voltage of approximately **22.2 V DC**.
- **Purpose:** This high-voltage configuration ensures sufficient headroom for voltage regulation and supports high-current loads such as motors and pumps.

2.2. Main Power Bus

- The combined 22.2 V output is routed to a **main power bus**, which acts as the central distribution node.
- All downstream components draw power from this bus, either directly or through regulated converters.

2.3. Motor Drivers and Actuators

- **DC Geared Motors (×4):** Four JGB37-545 DC geared motors are powered directly from the 22.2 V bus via Motor Driver 1 and Motor Driver 2. The front-left and back-left motors are wired in parallel to Driver 1, while the front-right and back-right motors are wired in parallel to Driver 2.
- **Water Pump:** The system utilizes a **24 V diaphragm pump**. It is powered directly from the **22.2 V Main Power Bus**, eliminating the need for a step-down converter while ensuring sufficient pressure and flow rate for fire suppression.

2.4. Voltage Regulation and Logic Power

- **Logic Step-Down Converter (XL4015):**
    - Converts 22.2 V to **5 V DC** for low-voltage components.
    - Powers:
        - **Two Servo Motors** (5 V each)

- **Converter Type:**
    - The system utilizes a single **XL4015 DC-DC Buck Converter**, chosen for its high efficiency to regulate the 5V logic rail.

2.5. Grounding Scheme

- A **common ground** is shared across all components to ensure stable voltage references and prevent floating ground issues.
- Ground lines are clearly marked and routed to maintain signal integrity and safety.

2.6. Design Considerations

- **Voltage Matching:**
    - Each component receives voltage tailored to its operating range via dedicated converters.
- **Current Handling:**
    - Motor drivers and converters are rated to handle peak currents without overheating or voltage sag.
- **Safety:**
    - Over-voltage, reverse polarity, and short-circuit protection are integrated into converters and drivers.
- **Modularity:**
    - The design allows easy replacement or upgrade of individual modules without affecting the entire system.

2.7. Summary of Voltage Paths

Source Voltage	Destination	Regulated Voltage	Components Powered
22.2 V (Series Li-Po)	Motor Driver 1 & 2	22.2 V	4x JGB37-545 DC Geared Motors
22.2 V	Relay Module	22.2 V	Water Pump(24V)

Source Voltage	Destination	Regulated Voltage	Components Powered
22.2 V	Step-Down Converter (XL4015)	5 V	Servo Motors
External Power Bank	Raspberry Pi 4 Model B	5.1 V	Raspberry Pi 4

This circuit design ensures robust power delivery across mechanical and computational subsystems, enabling reliable operation of the robotic platform. It reflects careful planning in voltage regulation, current distribution, and modular integration—key principles in embedded system design.

2.8. Power Distribution System

The robot's power system is engineered around a **22.2 V main bus**, derived from two 11.1 V Li-Po batteries connected in series. This configuration provides:

- High voltage headroom for efficient buck conversion
- Stable supply for high-current motors
- Reduced current draw across wiring, minimizing losses.

2.8.1. Voltage Rails

22.2 V Main Bus

- Directly powers the four high-torque JGB37-545 DC geared motors via dual BTS7960 drivers.
- Directly powers the 24 V high-pressure water pump.
- Provides input to the 5V buck converter.

5 V Rail (Step-Down Converter (XL4015))

- Powers:
    - Pan-tilt servos
- Isolated from motor loads to prevent brownouts.

This layered power system ensures stable operation across all subsystems, even during high-current events such as motor stall or pump activation.

3. Chapter 3: Distributed System Architecture

3.1. Executive Summary of Electrical Architecture

The Intelligent Mobile Robot operates on a distributed, two-node architecture designed to balance computational load, power efficiency, and real-time responsiveness. The system separates high-intensity computer vision tasks from onboard navigation and actuation, enabling reliable performance under battery-powered constraints.

The architecture consists of:

- **Node 1—Vision & Processing Station (Laptop):** Performs heavy computer vision tasks using custom CNN models for dual fire and human detection, and streams prioritized target coordinates to the robot.
- **Node 2 — Mobile Edge Node (Raspberry Pi 4 Model B):** Handles navigation, LiDAR processing, actuator control, and real-time decision-making.
- **Communication Layer — Local Mosquitto MQTT Broker:** Provides asynchronous, low-latency messaging between the two nodes.

This distributed design ensures that the robot maintains high responsiveness while minimizing onboard power consumption and thermal load.

3.2. Split-Node Operational Model

3.2.1. Vision Node (Stationary Laptop)

- Runs the custom CNN object detection pipelines (Fire Detection & Human Priority overriding).
- Receives RTSP video stream from the TP-Link Tapo C210
- Detect fire and compute its location within the environment.
- Publishes target coordinates to the MQTT broker.

3.2.2. Mobile Edge Node (Raspberry Pi 4 Model B)

- Subscribes to MQTT topics for navigation and fire-location data.
- Processes LiDAR readings for obstacle detection

- Controls motors, servos, pump, and solenoid valve
- Executes the fire-suppression sequence autonomously.

3.2.3. Communication Layer

- **local MQTT Broker (Mosquitto):** Replaces the previous cloud-based HiveMQ setup to handle asynchronous messaging directly over the local Wi-Fi or Ethernet network.
- **Ultra-Low Latency & Reliability:** By eliminating internet dependency and cloud routing, the system ensures near-instantaneous message delivery. This is critical for safety mechanisms (like the heartbeat watchdog) and real-time navigation commands.
- **Topic-Based Publish/Subscribe Structure:** Retains a highly modular communication format, allowing the vision node, navigation client, and hardware controllers to operate independently and making future system expansions simple.
- **Fully Localized Autonomy:** This updated architecture transitions the robot from a hybrid, cloud-dependent IoT setup into a fully localized edge-computing platform, maximizing real-time responsiveness and operational stability in environments without internet access.

Task / Feature	Raspberry Pi 4 Model B (Mobile Edge Node)	External Laptop (Vision Node)
Primary Role	Acts as the mobile onboard controller and edge-processor.	Acts as the stationary global vision and monitoring node.
Perception & Sensors	Handles local navigation and obstacle detection using the Okdo LiDAR HAT. It also manages the onboard Pi Camera V3.	Handles global perception by receiving and analyzing the RTSP video stream from the fixed TP-Link Tapo C210 camera.
Artificial Intelligence	Offloads heavy visual processing to save power and thermal load.	Runs the heavy custom CNN-based computer vision pipelines to identify fire sources and detect humans actively in danger.
Spatial Calculation	Runs the ROS 2 Navigation Stack (Nav2) and processes LiDAR data to generate local, collision-free paths to the target.	Performs Extrinsic Transformation calculations (using OpenCV and ArUco markers) to translate the 2D fire image into 3D physical coordinates.

Actuation & Hardware	Physically controls the hardware. It sends PWM signals to the BTS7960 motor drivers, controls pan-tilt servos, and triggers the 24V water pump via a relay.	Has no direct control over the robot's physical movement or suppression hardware; its role is purely computational.
MQTT Communication	Acts as the **Subscriber**. It listens to the local MQTT broker for incoming fire coordinates.	Acts as the **Publisher**. Once it calculates where the fire is, it sends that JSON coordinate payload to the local MQTT broker.

3.3. Computing and Control Hardware

3.3.1. Raspberry Pi 4 Model B — Mobile Control Unit

The Raspberry Pi 4 Model B serves as the robot's onboard controller, responsible for:

- Maintaining Wi-Fi connectivity
- Subscribing to MQTT topics
- Processing LiDAR data
- Generating PWM signals for motor drivers
- Controlling servos and relays
- Executing navigation and suppression logic

Its quad-core Cortex-A72 CPU and 4 GB RAM provide ample performance for real-time robotics tasks.

3.3.2. Laptop — Vision Processing Unit

The external laptop handles:

- Custom CNN-based fire and human detection.
- Frame-by-frame analysis of the Tapo C210 RTSP stream.
- Coordinate extraction and transformation.
- Publishing fire-location data to the MQTT broker

Offloading vision processing significantly reduces power consumption and thermal load on the robot.

4. Chapter 4: Actuation, Perception and Interface Logic

4.1. Actuators

4.1.1. Locomotion System

- **Motors:** Four JGB37-545 high-torque DC geared motors.
- **Drivers:** Dual BTS7960 H-Bridge modules
- **Voltage:** Powered directly from the 22.2 V main bus
- **Control:** PWM signals from Raspberry Pi GPIO pins

The BTS7960 drivers support up to 43 A peak current, making them ideal for high-load robotics applications.

4.1.2. Steering System

- **Mechanism:** 4-Wheel Skid-Steer
- **Actuator:** Driven entirely by the main JGB37-545 locomotion motors.
- **Function:** Directional control is achieved by varying the speed and rotational direction of the left and right motor banks independently, generating the lateral friction required to turn the chassis.

4.1.3. Fire Suppression System

- **Pump:** 24 V diaphragm pump (4 L/min)
- **Aiming:** Pan-tilt mechanism using MG996R metal-gear servos
- **Trigger:** Solenoid valve controlled via opto-isolated relay

This system enables accurate targeting and controlled water discharge.

4.2. Sensors and Perception Layer

4.2.1. Global Detection — TP-Link Tapo C210

- Provides 2K video stream via RTSP.
- Mounted in a fixed position for full-room coverage.
- Acts as the “global eye” for fire detection

4.2.2. Local Navigation — Okdo LiDAR HAT

- DTOF-based 360° scanning.
- Mounted directly on the Raspberry Pi GPIO header
- Provides real-time obstacle detection and mapping.
- Enables safe navigation toward target coordinates.

4.3. Communication and Logic Flow

4.3.1. MQTT Workflow

1. **Detection:** Laptop detects fire and evaluates human presence using custom CNNs.
2. **Localization:** Coordinates are computed
3. **Publishing:** Laptop publishes JSON payload to local broker
4. **Subscribing:** Raspberry Pi receives the message
5. **Execution:** Robot navigates and activates suppression system

This asynchronous communication ensures low latency and high reliability.

4.4. Interface and Wiring Logic

This section describes the detailed wiring strategy used to interface the Raspberry Pi 4 Model B with the motor drivers, LiDAR module, servos, pump, and solenoid valve. The design prioritizes electrical safety, logic-level compatibility, and reliable high-current operation.

4.4.1. Raspberry Pi 4 Model B GPIO Allocation

The Raspberry Pi 4 Model B serves as the central control unit for all low-voltage logic signals. To ensure electrical isolation from motor noise, the Pi is powered independently via an external USB-C power bank. **Note:** A shared ground line between the Pi and the main motor breadboard is strictly required to ensure stable PWM signal reference.

GPIO Pin Mapping Table

Component	Pin Name	Connection Point	Physical Pin #
Pi Power (Main)	USB-C Port	External Power Bank	N/A
	5V	DISCONNECTED	Pin 2
	GND	Breadboard GND Rail	Pin 6
	3.3V	Breadboard 3.3V Rail	Pin 1
Left Motor Driver (BTS7960)	VCC	Breadboard 5V Rail (XL4015)	
	GND	Breadboard GND Rail	
	R_EN	3.3V Rail	
	L_EN	3.3V Rail	
	R_PWM	GPIO 17	Pin 11
	L_PWM	GPIO 27	Pin 13
Right Motor Driver (BTS7960)	VCC	Breadboard 5V Rail (XL4015)	
	GND	Breadboard GND Rail	
	R_EN	3.3V Rail	
	L_EN	3.3V Rail	
	R_PWM	GPIO 25	Pin 22
	L_PWM	GPIO 23	Pin 16
LIDAR HAT	5V Power (blue)		Pin 4
	GND (red)		Pin 34
	TX (white)		Pin 10
	PWM (yellow)		Pin 8
Pump Relay Module	PUMP_CONTROL	GPIO 26	Pin 37
MG996R Servo (180°)	VCC (red)	Breadboard 5V Rail (XL4015)	
	GND (brown)	Breadboard GND Rail	

	Signal (orange/yellow)	GPIO 13	Pin 33
MG996R Servo (360° continuous)	VCC (red)	Breadboard 5V Rail (XL4015)	
	GND (brown)	Breadboard GND Rail	
	Signal (orange/yellow)	GPIO 18	Pin 12

4.4.2. Logic-Level Compatibility and Protection

The Raspberry Pi 4 Model B operates strictly at **3.3 V logic**, requiring careful interfacing with higher-voltage components.

Motor Driver Enable Logic

The BTS7960 motor drivers accept 3.3 V as a valid HIGH signal. To simplify control and reduce GPIO usage:

- **R_EN and L_EN pins are hard-wired to the Pi's 3.3 V rail.**
- This keeps both H-bridge channels permanently enabled.
- The Pi only needs to send PWM signals to control direction and speed.

This approach reduces software complexity and ensures the drivers are always ready to receive commands.

4.4.3. Unified Grounding Strategy

PWM Control Signals

The Raspberry Pi directly drives the BTS7960 PWM pins:

- **R_PWM** → Forward rotation
- **L_PWM** → Reverse rotation

The BTS7960's internal opto-isolation and logic circuitry safely accept 3.3 V PWM signals without level shifting.

Relay Control for Pump and Solenoid

The water pump and solenoid valve are inductive loads. To protect the Raspberry Pi:

- Both are controlled using **opto-isolated relay modules**.
- The relay input side uses 3.3 V logic from the Pi.
- The output side switches the **22.2 V Main Bus** to the pump.
- Flyback protection is handled inside the relay module.

LiDAR HAT Interface

The Okdo LiDAR HAT mounts directly onto the Raspberry Pi's 40-pin header.

- Uses **UART** communication.
- Draws power from the Pi's 5 V and 3.3 V rails.
- No external wiring required.
- Eliminates cable clutter and reduces signal noise.

This direct-mount strategy ensures stable communication and simplifies assembly.

4.4.4. High-Current Wiring and Noise Mitigation

Grounding Strategy

A unified grounding scheme is used across the entire robot:

- All converters, drivers, relays, and Raspberry Pi share a **common ground.**
- This prevents floating reference levels.
- Ensures accurate PWM signaling.
- Reduces electrical noise from motors and pump.

Proper grounding is essential for stable operation of mixed-voltage systems.

High-Current Wiring Considerations

Because the robot uses high-current motors and a **24 V pump**:

- Thick-gauge wires are used for the 22.2 V main bus.

- Signal wires are routed separately from power wires to avoid interference.
- The pump is wired to the main bus closest to the battery source to minimize voltage drop affecting the logic converter.

5. Chapter 5: Kinematic and Transformation Modeling

The physical platform utilizes a 4-wheel drive system actuated by two BTS7960 motor drivers. The left motors (front and rear) are driven in parallel, as are the right motors. Consequently, the mechanical motion of the platform is governed by the **Skid-Steer Kinematic Model**.

Unlike standard differential drive systems, turning a skid-steer chassis requires overcoming lateral friction, causing the wheels to slip across the ground.

5.1. Skid-Steer System Parameter Initialization

Based on the physical dimensions established in the robot's unified robot description format (URDF):

- **Track Width (B):** The lateral distance between the centerlines of the left and right wheels is 0.35 meters (calculated from the y-axis offsets of 0.175 and -0.175).
- **Wheelbase** (L): The longitudinal distance between the front and rear wheels is 0.40 meters (calculated from the x-axis offsets of 0.2 and -0.2).
- **Wheel Radius (R):** 0.065 meters.

While these parameters $(B = 0.35\text{m}, L = 0.40\text{m})$define the calibrated URDF digital twin profile optimized for simulation stability within Gazebo Harmonic , they are dynamically adjusted within the physical embedded ROS 2 hardware control drivers to map finalized physical chassis configurations $(B \approx 0.40\text{m}, L = 0.47\text{m})$.

5.2. Inverse Kinematics (Velocity Commands to Wheel Speeds)

Inverse kinematics determines the necessary rotational velocities of the left and right wheel sets (v_L) and (v_R) to achieve a desired linear velocity (v) and angular velocity (ω) for the robot chassis.

Assuming ideal conditions where the front and rear wheels on each side rotate at the identical speed ($v_{front\_left} = v_{rear\_left} = v_L$),the equations are:

$$v_L = v - \frac{\omega \cdot B}{2}$$

$$v_R = v + \frac{\omega \cdot B}{2}$$

To convert these linear tangential wheel speeds to the angular velocity of the motors (ω_m, measured in radians per second):

$$\omega_{L\_motor} = \frac{v_L}{R}$$

$$\omega_{R\_motor} = \frac{v_R}{R}$$

5.3. Forward Kinematics (Wheel Speeds to Robot Motion)

Forward kinematics calculates the robot's resulting linear and angular velocities (v and ω) in the world frame based on the current velocities of the left and right wheel sets (v_L and v_R).

To accurately model a skid-steer robot in the real world, an empirical **slip factor (c)** is introduced to account for the efficiency lost to lateral friction during a turn.

$$v = \frac{v_R + v_L}{2}$$

$$\omega = \frac{v_R - v_L}{B \cdot c}$$

The robot's pose (x, y, θ) is estimated by integrating these velocities over time:

$$x(t) = \int v \cdot \cos(\theta)\, dt$$

$$y(t) = \int v \cdot \sin(\theta)\, dt$$

$$\theta(t) = \int \omega dt$$

5.4. Homogeneous Transformation Matrices (TF)

Transformation matrices define the precise spatial relationships (translation and rotation) between the center of the robot (base_link) and its various sensors and actuators. These are modeled as 4×4 homogeneous matrices (T):

$$T = \begin{bmatrix} R_{3\times3} & P_{3\times1} \\ 0 & 1 \end{bmatrix}$$

- **Base to Okdo LiDAR**

The lidar_joint connects the lidar_link to the base_link with a pure translation of 0.21 meters along the z-axis and zero rotation.

$$T_{base}^{lidar} = \begin{bmatrix} 1 & 0 & 0 & \text{-}0.120 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0.21 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

- **Base to Pi Camera V3**

The camera_joint connects the camera_link to the base_link with a translation of 0.265 meters on the x-axis, 0.228 meters on the z-axis, and zero rotation.

$$T_{base}^{camera} = \begin{bmatrix} 1 & 0 & 0 & 0.265 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0.228 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

- **Base to Front-Left Wheel**

The front_left_wheel_joint involves both translation and rotation. It is translated 0.2 meters (x) and 0.175 meters (y) from the base. To align the wheel cylinder properly, it is rotated -1.5708 radians ($-90°$) around the x-axis (Roll).

Using the standard rotation matrix for the x-axis, where $\cos(-90°) = and \sin(-90°) = -1$:

$$T_{base}^{FL\_wheel} = \begin{bmatrix} 1 & 0 & 0 & 0.2 \\ 0 & 0 & 1 & 0.175 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

5.5. Extrinsic Transformation Modeling (Surveillance Camera to Robot Frame)

To achieve full autonomy, the system utilizes an external surveillance camera running a custom CNN vision processing node to detect flames and prioritize human safety. To bridge the camera's fixed coordinate frame with the mobile robot's coordinate frame, an extrinsic transformation matrix (T_{camera}^{robot}) must be computed dynamically in real-time.

This mapping is achieved using fiducial markers (specifically, ArUco markers) mounted securely on the robot chassis. By detecting the known geometric properties of the marker, the vision system solves the Perspective-n-Point (PnP) problem to extract the physical translation vector ($tvec$) and rotation vector ($rvec$) of the robot relative to the camera lens.

The rotation vector is converted into a 3×3 rotation matrix (R) using Rodrigues' rotation formula. The resulting components are combined to construct the 4×4 homogeneous transformation matrix:

$$T_{camera}^{robot} = \begin{bmatrix} R_{11} & R_{12} & R_{13} & tvec_x \\ R_{21} & R_{22} & R_{23} & tvec_y \\ R_{31} & R_{32} & R_{33} & tvec_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Calculating this matrix allows the system to map the 2D pixel coordinates of the detected flame into the physical 3D workspace. This spatial data is then transmitted to the robot via the local MQTT broker, enabling the ROS 2 Navigation Stack to compute a precise path to the target.

5.6. Vision Node Software Integration and Implementation Overview

The external Vision Node (Laptop Processing Unit) serves as the high-level computational engine for the distributed framework. Because the local mobile edge unit (Raspberry Pi 4 Model B) operates under severe power constraints and thermal limits , intensive computer vision calculations are completely offloaded to the external node over a localized network.

The vision software script is engineered around three core paradigms: asynchronous, multithreaded video frame acquisition; real-time geometric coordinate translation via

fiducial marker tracking; and a tiered neural network inference hierarchy. To ensure modular access across the system, a lightweight web framework is integrated directly into the script to broadcast processed tracking visualizations.

5.6.1. Asynchronous Multithreaded Frame Capture Architecture

A common bottleneck in real-time robotics vision is the internal buffer saturation of standard video capture systems (such as OpenCV's VideoCapture class). When executing computationally heavy deep learning models, the latency introduced by model inference causes the frame buffer to backup, resulting in a severe telemetry lag between physical movements and visual perception.

To mitigate this deficiency, the software implements a dedicated frame drainage engine utilizing Python's threading library. The architecture segregates frame grab mechanisms from processing blocks as follows:

- **The Background Worker Thread:** Upon initialization, a separate worker thread is spawned to run a continuous, non-blocking hardware loop. This loop constantly drains the incoming network video stream buffer at maximum framerate. It intentionally overwrites older frames in the memory space, maintaining exclusively the single most recent, uncorrupted frame captured by the lens.
- **The Main Processing Call:** When the central AI pipeline requests a frame for analysis, it reads directly from the thread's active memory reference rather than awaiting hardware buffer extraction. This ensures near-instantaneous frame delivery ($< 10\ ms$ processing latency) and decouples visual input generation from neural network execution speeds.

5.6.2. Deep Learning Pipeline and Network Topography

The Vision Node simultaneously executes two separate deep learning frameworks over distinct camera streams to orchestrate safety and localization behaviors. The software handles this using a multi-tiered model hierarchy:

1. Global Surveillance Tier (GFD-Net via PyTorch)

The software connects to the primary wide-area environmental camera (TP-Link Tapo C210) via the Real-Time Streaming Protocol (RTSP). The incoming 2K frames are

downsampled and converted into a standard normalized 3-channel RGB matrix matching the inputs expected by the custom **Global Fire Detection Network (GFD-Net)**.

The **GFD-Net** framework processes each frame by passing it through a sequential network backbone of 2D convolutional layers, rectified linear unit (ReLU) activations, and max-pooling stages to downsample spatial dimensionality while distilling deep feature maps. An adaptive average pooling layer standardizes the grid spatial map to a structural array, after which a specialized convolutional detection head predicts bounding box coordinates, class labels, and localized activation confidence matrices.

The script parses these maps to isolate the highest confidence detection grid. If the extracted confidence metric surpasses a hardcoded threshold (50%),the normalized grid anchors are mathematically scaled back into the original pixel dimensions of the raw surveillance image stream to overlay red flame-boundary bounding boxes.

2. Local Perception Tier (Keras Models via TensorFlow)

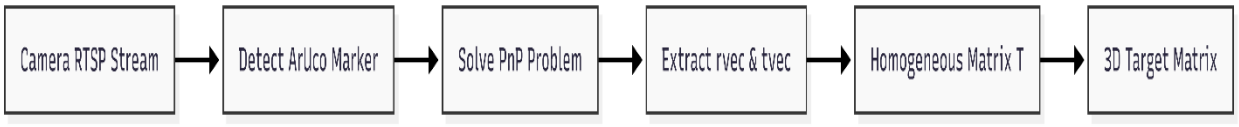
Simultaneously, the script acts as a diagnostic server for the robot's local perception layer. It acquires local video arrays streamed by the mobile edge node's Raspberry Pi Camera Module V3. The stream input is duplicated into identical matrices, resizing the dimensions to meet input conditions required by two binary classification structures built using Keras APIs.

- **Localized Fire Tracking:** The first Keras model continually tracks whether a fire source is actively within the immediate horizontal field of view of the chassis, transforming pixel matrices to track changes in immediate flame proximity.
- **Human Presence Overriding:** The second Keras model executes a specialized human-detection classifier. If a human profile is flagged within this local edge zone, it triggers an executive priority override. This safety mechanism forces the robot to prioritize life safety and hazard mitigation surrounding the human target over standard structural fire suppression goals.

5.6.3. Fiducial Marker Tracking and Coordinate Extraction Logic

To bridge the gap between 2D digital image coordinates and the 3D physical workspace, the vision node processes fiducial marker tracking in tandem with the neural network arrays. The software leverages OpenCV’s ArUco marker library, targeting a predefined 4x4

square tracking marker array mounted structurally to the upper surface of the mobile chassis.



The algorithm performs localization through the following workflow:

1. **Geometric Space Definition:** The script initializes an array defining the exact physical coordinates of the tracking marker's corners in its own local coordinate system based on empirical measurements of the physical marker (9.3 cm).
2. **Perspective-n-Point (PnP) Resolution:** Upon visual detection of the marker's corners within a frame, the coordinates are cross-referenced with pre-calibrated internal camera matrix parameters and distortion coefficients. The software feeds these constraints into an iterative Perspective-n-Point solver utilizing the SOLVEPNP_IPPE_SQUARE flag, which optimizes orientation accuracy for planar structural markers.
3. **Transformation Matrix Assembly:** The PnP solver outputs a 3D translation vector $tvec$ and a 3D rotation vector $rvec$ describing the exact offset of the robot relative to the camera lens optical center. The script converts the rotation vector into a 3×3 rotation matrix (R) via Rodrigues' rotation formula. These variables are compiled into a unified 4×4 homogeneous transformation matrix (T_{camera}^{robot}):

4. $$T_{camera}^{robot} = \begin{bmatrix} R_{11} & R_{12} & R_{13} & tvec_x \\ R_{21} & R_{22} & R_{23} & tvec_y \\ R_{31} & R_{32} & R_{33} & tvec_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Coordinate Export: By resolving this matrix, the exact center of the robot is tracked in continuous 3D coordinate space (X, Y).When the GFD-Net global network flags a fire pixel centroid, this transformation matrix maps the fire's pixel vector onto the physical ground plane grid. The resulting real-world coordinate position payload is formatted into a standardized JSON string and dispatched over a local Mosquitto MQTT broker network to instruct the ROS 2 Navigation stack.

5.6.4. Flask Video Telemetry Microserver

To allow teleoperation and monitoring by human operators, the script runs a background Flask web server concurrent with the core computer vision loops. The server exposes dedicated HTTP network routes (e.g., /video_feed_tapo and /video_feed_pi) protected by Cross-Origin Resource Sharing (CORS) rules to enable remote dashboard connections.

The visualization script continually compresses the fully annotated OpenCV frame matrices—complete with red bounding boxes tracking fire targets and classification text labels tracking Keras predictions—into JPEG streams. These frames are served via a multipart HTTP response protocol utilizing an x-mixed-replace boundary mime-type. This structure allows the user browser connection to continuously stream the changing frame arrays as a live video feed, presenting real-world telemetry with minimal data transmission overhead.

6. Chapter 6: ROS 2 Software Integration and Control Nodes

6.1. Phase 1: Simulation and Robot Modeling (Gazebo Harmonic & WSL Environment)

The initial phase of the software validation workflow focuses on constructing a precise digital twin of the physical robotic platform and establishing a high-fidelity virtual proving ground. Rather than relying strictly on immediate empirical hardware deployments, the system's spatial, inertial, and sensor properties—alongside its deployment workspace—are formalized into an integrated simulation layer and instantiated within the physics-enabled Gazebo Harmonic environment.

Because the core development stack operates within a Windows Subsystem for Linux (WSL) container, specific rendering abstractions (such as the OGRE engine) and strict simulation parameters are enforced to ensure graphics pipeline stability and deterministic physics execution.

To ensure full compatibility with the Rigid Body Dynamics algorithms embedded in the ROS 2 Navigation Stack (Nav2) and KDL (Kinematics and Dynamics Library) parsing engines, the structural link hierarchy originates at an empty, zero-mass coordinate origin defined as base_footprint. This link serves as the projection of the robot's physical center onto the 2D ground plane workspace. A fixed joint connects this footprint to the primary structural link, base_link, tracking spatial orientations without adding unnecessary transformation complexities to the runtime coordinate tree.

6.1.1. Structural Chassis Geometry and Inertial Matrix Mapping

The core structural architecture of the robot description defines the physical dimensions, safety collision bounds, and mass distribution parameters of the physical chassis:

- **Chassis Volume:** The physical chassis envelope is modeled as a rigid rectangular box link measuring exactly $0.465\ meters$ in length $(x - axis)$, $0.355\ meters$ in width $(y - axis)$, and $0.200\ meters$ in height $(z - axis)$. This bounded volume defines the

primary visual boundary and collision check bounds utilized by the Nav2 costmap layers to prevent structural clipping against obstacles.

- **Mass and Inertia:** The structural link specifies a total chassis mass parameter of $5.4\ kilograms$ centered at a local vertical offset $z = 0.100\ meters$. To allow realistic torque responses, slippage, and acceleration profiles within the Gazebo physics engine, a full 3×3 rotational inertia tensor matrix is calculated and bound to the link structure. The primary moments of inertia are specified as $I_{xx} = 0.07471$, $I_{yy} = 0.11530$, and $I_{zz} = 0.15402$ kg $\cdot$ m^2. This asymmetric distribution reflects the true physical component placement, ensuring proper longitudinal stability during sudden velocity adjustments.

6.1.2. Locomotion and Steering Kinematics Parameterization

The mobile drivetrain uses a 4-wheel drive configuration, governed by Skid-Steer Kinematic principles. Within the description layout, the four wheels are segregated into individual visual and collision links connected to the core base_link via continuous rotational joints:

- **Wheel Dimensions:** Each locomotion wheel link is modeled as a standardized cylinder featuring a physical radius R of $0.065\ meters$ and an absolute thickness (length) of $0.05\ meters$. Each individual wheel link carries an independent mass profile of $0.2\ kilograms$ and localized inertia values ($I_{xx} = 0.00025, I_{zz} = 0.00042$ kg $\cdot$ m^2) to calculate rotational wheel inertia correctly.
- **Spatial Track and Wheelbase:** The joint positions map the exact kinematic footprint of the platform. The longitudinal wheelbase L measures $0.40\ meters$, derived from front-to-rear center offsets spanning from $x = 0.2\ m$ to $x = -0.2\ m$. The lateral track width B spans $0.35\ meters$, matching the physical distance separating the wheel centerlines along the y-axis (offsets of ± 0.175 m).
- **Asymmetric Contact Friction Surfaces:** To accurately replicate skid-steer lateral turning friction inside a simulator, custom contact friction surfaces are assigned to each wheel link description. The primary slip friction parameter (μ_1)is set to a high value of 1.0 along the forward direction of travel to ensure strong linear traction. Crucially, the secondary lateral slip coefficient (μ_2) is reduced to an empirical factor of 0.05. This targeted reduction minimizes turning resistance, allowing the wheels to smoothly slip sideways across the ground plane during angular commands without destabilizing or tipping the platform.

6.1.3. Sensor and Actuator Coordinate Link Transforms

To resolve incoming distance coordinates and target visuals into standard base metrics, the description file establishes precise spatial transformations T linking perception sensors directly to the physical origin of the robot:

- **Base-to-LiDAR Link** (T_{base}^{lidar})**:** The lidar_link describes the physical housing of the Okdo LiDAR HAT. It is joined to the center of the chassis via a fixed joint tracking a pure vertical translation offset of exactly 0.21 metersalong the positive z-axis, with zero angular orientation error. This rigid relationship is modeled via a clean homogeneous transformation matrix:
- $T_{base}^{lidar} = \begin{bmatrix} 1 & 0 & 0 & \text{-}0.120 \\ 0 & 1 & 0 & 0.00 \\ 0 & 0 & 1 & 0.21 \\ 0 & 0 & 0 & 1 \end{bmatrix}$

  Base-to-Camera Link (T_{base}^{camera})**:** The local perception module tracking the Raspberry Pi Camera Module V3 is mapped via the camera_link element. Positioned at the leading edge of the robot chassis, its fixed joint tracks a forward translation of 0.2325 meters along the x-axis and a vertical clearance offset of 0.100 meters along the z-axis. This transformation handles immediate parallax corrections when converting pixel objects to local distance points.

6.1.4. Custom Warehouse Simulation Environment Layout

To rigorously validate the SLAM (Simultaneous Localization and Mapping) and autonomous navigation stacks, a dedicated spatial environment model was constructed. The simulation world establishes a virtual $20 \times 20\ meter$ bounded industrial layout representing a high-risk multi-level warehouse facility.

The physics constraints within this world model enforce a fixed time step simulation rate $(max\_step\_size = 0.001\ \text{s})$ matching a real-time factor execution target of 1.0 to ensure behavioral consistency. The layout consists of structural perimeter containment walls $(North, South, East, West)$ defining absolute map constraints. To challenge the robot's sensor perception and path generation nodes, the internal floor layout is populated with narrow industrial shelving aisles. These shelving units are modeled as rigid blocks $12\ m \times 1\ m \times 2\ m$ placed at parallel offsets $(y = 3\ \text{m}\ and\ y = -3\ \text{m})$.

Finally, a permanent target—modeled as a static 1 meter solid red box—is positioned in the back quadrant $x = 8\,m, y = 8\,m$. This object serves as a permanent environmental target during testing, allowing the developer to verify that the navigation stacking stack can successfully compute collision-free global paths through narrow corridors.

6.1.5. Embedded Gazebo Sensor and Drive Plugins

To bridge the static structural model with dynamic runtime processes, the simulation ecosystem utilizes integrated Gazebo simulator extension blocks:

1. **Drivetrain Control Plugin (gz-sim-diff-drive-system):** This plugin binds the continuous wheel joints into a singular logical locomotion module. It intercepts linear and angular velocity vectors directly from the standard global /cmd_vel topic, passing calculations through the track and radius constraints ($wheel\_separation = 0.35$ m, $wheel\_radius = 0.065$ m) to drive the virtual chassis. The driver calculates and broadcasts continuous real-time odometry metrics (/odom) and transform frames (/tf) at an update frequency of 30 Hz.
2. **LiDAR Simulation Sensor Node (gpu_lidar):** Binds a virtual laser scanner directly onto the physical coordinate center of the lidar_link frame. The sensor simulates ray-cast arrays tracking a full $360°$ horizontal field of view $6.28319\ radians$ split into 360 discrete sample points at an update rate of 10 Hz. The ray range matrix matches physical DTOF limits, calculating distance fields from a minimum clearance of 0.35 meters out to a maximum range of 10.0 meters to generate virtual occupancy grids.
3. **Optical Camera Simulation Node (camera):** Instantiates an active image matrix on the camera_link structural position. The sensor captures simulated environment visuals across a horizontal field of view of 1.309 radians $(75°)$, exporting standard 640×480 pixel resolution arrays at 30 frames per second onto the global phoenix/camera/image_raw topic.

6.2. Phase 2: Hardware Control (Raspberry Pi 4 Model B Embedded Driver)

The second phase of the software integration package establishes the fundamental low-level abstraction bridge that translates continuous kinematic control trajectories generated in the virtual environment or navigation stack into physical electrical signals. Operating

natively on the onboard edge-computing microprocessing core (Raspberry Pi 4 Model B), the hardware abstraction layer manages real-time, non-blocking digital I/O execution, pulse-width modulation (PWM) delivery, and inductive protection loops.

To align with structural ROS 2 guidelines, these embedded drivers are encapsulated within a dedicated execution workspace package (phoenix_control), designed to directly interface with structural motor configurations. This architectural encapsulation completely isolates algorithmic navigation data layers from real-time bare-metal pin interactions.

6.2.1. Velocity Message Subscription and Interprocess Transport

The entry point of the embedded hardware control infrastructure centers around an active subscriber loop hooked directly into the intraprocess communications network. The control node instantiates a standardized ROS 2 sub-routine configured to dynamically sample incoming geometry_msgs/msg/Twist payload data streamed over the central high-speed communication channel (/cmd_vel).

The incoming data frame splits into two primary spatial variables:

- **Linear Translation Vector** $(\dot{x})$**:** Extracted from msg.linear.x, dictating the target forward or backward movement speed of the robot chassis along its longitudinal trajectory.
- **Angular Yaw Vector** $(\dot{\theta}_z)$**:** Extracted from msg.angular.z, defining the target rotational velocity commands required to alter the orientation heading of the robot platform.

By updating these values asynchronously inside an optimized interrupt handler callback routine, the control infrastructure eliminates polling latency, ensuring the embedded hardware driver responds instantly ($20\ Hz$ execution loop frequency) to volatile variations in navigational goal states.

6.2.2. Velocity Smoothing and Target Approach Algorithms

Direct, unsmoothed execution of raw velocity commands onto high-torque DC geared motors can cause severe systemic failures, including mechanical gear stripping, voltage sags on the main power rail, and tracking drift due to sudden wheels slippage. To address this risk, the embedded control loop incorporates an active interpolation velocity smoothing layer.

The algorithm uses continuous conditional logic loops that run sequentially every 0.05 seconds to step current velocities toward the targets. The logic operates using hardcoded acceleration increments:

$$\Delta v_{\text{linear}} = 0.05 \quad \text{and} \quad \Delta \omega_{\text{angular}} = 0.05$$

Every 50 ms, the approach routine compares the currently executing velocity state with the fresh target message. If the active speed falls below the target threshold, the system mathematically increments the output by the fixed step factor, capping it via a min() boundary rule to prevent target overshoot. Conversely, if the active command exceeds the required threshold, the value decrements by the step factor utilizing a max() boundary rule. This smooth interpolation creates a predictable ramp profile, guaranteeing it takes exactly 1.0 second to scale from a dead stop up to maximum output velocity $1.0\ unit$, protecting structural elements from high transient current draws.

6.2.3. Differential-to-Hardware Kinematic Mapping and Calibration Subroutines

Once the linear and angular target matrices are fully smoothed, the node processes a hardware-centric kinematic calculation loop to resolve individual wheel bank speeds:

$$V_{\text{left}} = V_{\text{smoothed}} - \Omega_{\text{smoothed}}$$
$$V_{\text{right}} = V_{\text{smoothed}} + \Omega_{\text{smoothed}}$$

Because the physical robot platform is structured as an integrated 4-wheel drive skid-steer architecture with left and right wheel sets wired symmetrically in parallel to dual BTS7960 high-current H-bridge motor drivers, the hardware abstraction maps these computed wheel velocities into distinct left and right command outputs.

To account for variances in manufacturing tolerances or accidental polarity reversals during physical wiring assembly, the initialization routine includes hardcoded behavioral configuration flags:

- **Spatial Swapping Subroutine (swap_left_and_right):** A logical boolean parameter that, when set to True, mathematically reverses the right and left control arrays to correct spatial turning errors.
- **Inversion Protection Subroutines (invert_left and invert_right):** Independent flags that toggle negative scale operations across computed wheel bank velocity

matrices, aligning directional software commands with actual physical motor rotations.

Following alignment adjustments, a normalization script screens the absolute wheel speeds to evaluate if any output exceeds physical constraints > 1.0. If an overload condition is flagged, both left and right speed channels are proportionally scaled down by the maximum overage value, preserving the exact target angular radius while protecting the drivers from over-saturation.

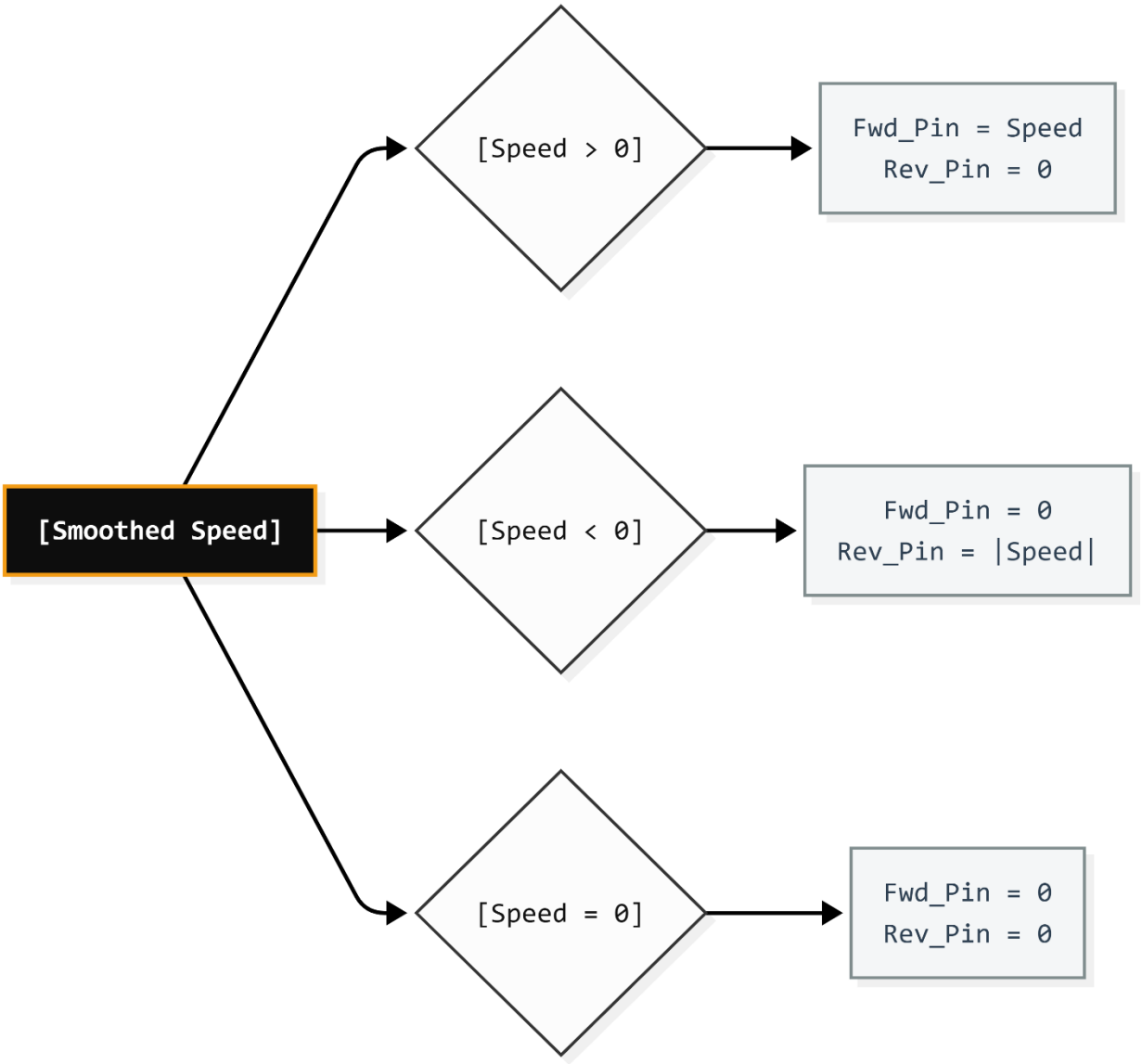
6.2.4. Pulse-Width Modulation (PWM) and H-Bridge Pin Abstraction

The final layer of the hardware control loop maps the normalized velocity matrices into physical output signals routed directly to the Raspberry Pi's Broadcom GPIO header array. Rather than calling lower-level memory registries manually, the architecture utilizes the optimized gpiozero object framework to command separate forward (R_PWM) and reverse (L_PWM) signal paths.

The physical H-Bridge control strategy maps to four discrete hardware interfaces:

1. **Left Forward Channel (GPIO 17 / Pin 11):** Transmits continuous high-frequency forward duty cycles to Motor Driver 1.
2. **Left Reverse Channel (GPIO 27 / Pin 13):** Transmits reverse direction travel duty cycles to Motor Driver 1.
3. **Right Forward Channel (GPIO 25 / Pin 22):** Transmits forward duty cycles to Motor Driver 2.
4. **Right Reverse Channel (GPIO 23 / Pin 16):** Transmits reverse direction travel duty cycles to Motor Driver 2.

The underlying software processing logic uses conditional branch routines to activate the specific direction paths:



- **Positive Velocity Profile $V > 0$:** The forward channel pin updates to match the continuous floating-point velocity scalar directly, while the paired reverse channel pin drops to a strict low logical state $0.0\ V$.
- **Negative Velocity Profile $V < 0$:** The forward channel pin drops to zero, while the reverse pin asserts a positive voltage equal to the absolute value of the computed negative speed string.

- **Neutral Velocity Profile $V = 0$:** Both forward and reverse channel pins dump their electrical charges to zero, bringing all locomotion subsystems to an immediate electrical lock state to prevent floating drift.

Finally, an integrated safety management hook wraps the node's lifecycle functions. If a keyboard interrupt or global system failure is intercepted by the ROS 2 worker thread, the runtime wrapper overrides all active control matrices, forcing every locomotion channel pin to zero output. This fail-safe mechanism safely shuts down the physical 22.2V motor circuitry before the node container is fully unmounted and destroyed.

6.3. Phase 3: Actuator, Sensor, and Communication Nodes

To seamlessly bridge the raw embedded hardware layer with the high-level ROS 2 Navigation Stack (Nav2) and mapping utilities, Phase 3 establishes a distributed, multi-threaded node array on the Raspberry Pi 4 Model B. These asynchronous hardware-interface nodes handle physical sensor ingestion, stream translation, network coordination, and safe suppression actuation.

By operating within independent node execution containers, the architecture prevents localized blockages—such as hardware serial delays—from crashing or delaying core path-planning loops.

6.3.1. Fire Suppression Pump Control Infrastructure

The execution node responsible for local fire extinguishing acts as a hardwired relay driver configured to trigger upon arrival confirmation. The underlying software wrapper instantiates a standard subscriber pipeline listening continuously to a local boolean topic (target_reached).

The control logic implements an automated, interrupt-driven workflow:

- **Goal Status Verification:** The node remains in an absolute low-power state until an incoming message asserts a logical True flag. This flag is broadcast by the navigation action client only when the robot's physical coordinate envelope successfully hits the target tolerance threshold surrounding the tracked flame source.
- **Timed Execution Cycle:** Upon flag assertion, the node switches an active high-state command to the hardware-mapped Broadcom GPIO 26 pin. This digital output

instantly energizes an opto-isolated relay module. This module isolates sensitive 3.3V digital components from high-current inductive surges while closing the 22.2V main power circuit to fire up the 24V diaphragm water pump.

- **Safe Termination Loop:** To prevent accidental water damage to high-value assets or flooding within the workspace, a hardcoded sleep parameter limits the continuous spraying duration to exactly 5.0 seconds. Once this duration expires, an explicit termination override asserts a logical False state to drop the pin output to zero, cutting pump power. A fallback shutdown hook guarantees that if the node is terminated early by the user or encounters a network drop, the pin is forced low to ensure the pump fails safe.

6.3.2. UART LiDAR Frame Telemetry Parsing and Scanning Pipeline

The navigation and mapping stacks require an unbroken, low-latency stream of normalized distance information to build costmaps and prevent collisions. The LiDAR interface node serves as a high-speed serialization driver that transforms raw binary streams arriving via the Raspberry Pi's hardware UART serial port (/dev/ttyAMA0 at a baud rate of 230400) into standardized messages.

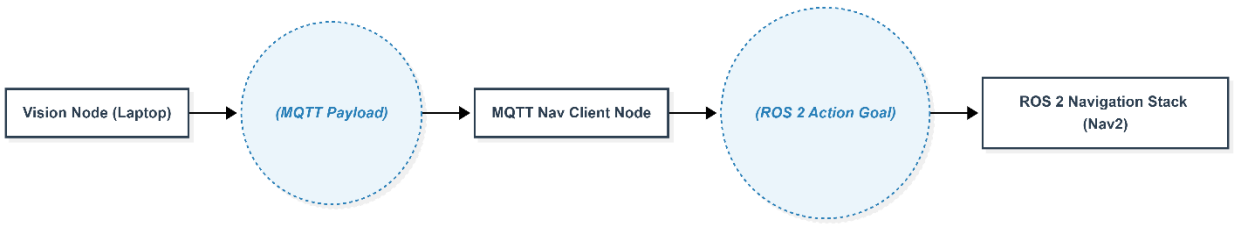
Because raw telemetry frames arrive as an unstructured byte stream, the node maintains a continuous internal ring buffer to perform multi-stage packet decoding:

1. **Header Synchronization:** The parsing algorithm continuously scans the byte buffer array to find specific multi-byte sequence markers matching the hardware specification: a sync header (0x54) combined with a fixed packet length byte (0x2C). If alignment is lost due to line noise, the node discards individual bytes sequentially until synchronization is re-established.
2. **Angular & Distance Resolution:** Once a valid 47-byte block is successfully locked, the algorithm extracts the starting and ending sweep angles from specific little-endian indices, scaling them down to true degrees. It then processes the 12 intermediate point distance blocks embedded inside the payload packet. The distances are translated from raw little-endian millimeter integers into true floating-point meters.
3. **Index Mapping and LaserScan Compilation:** Each resolved point distance is mapped into its nearest matching integer degree array slot $(0° \; to \; 359°)$ inside a

persistent 360-element range array. At a fixed rate of 10 Hz, this range matrix compiles into a standard sensor_msgs/msg/LaserScan message data payload. The frame origin header is stamped as lidar_link, bounding the field of view between 0 and $2\pi\ radians$ with a strict 1° resolution increment. A strict range filter is applied between 0.35 meters and 10.0 meters, automatically ignoring closer data points to prevent the robot's own physical chassis from triggering ghost obstacles on the global costmap.

6.3.3. Asynchronous MQTT to Nav2 Action Client Coordination Node

Serving as the primary communication bridge in the distributed split-node topography, this node acts as a translator between localized IoT message brokers and internal ROS 2 action interfaces. The node runs a persistent background Paho-MQTT client thread connected to the local Mosquitto host broker over network port 1883.



The data routing and path coordination sequence follows an asynchronous event-driven structure:

- **Network Target Acquisition:** The background thread subscribes directly to the navigation coordinates topic (ambers/robot/navigation/target). When the external vision node computes a fresh target coordinate vector, it publishes a compact JSON string payload containing real-world Cartesian location points (x and y). The client node intercepts this packet, unpacks the raw telemetry, and parses the fields into continuous floating-point numbers.
- **Nav2 Action Goal Dispatch:** The node converts these coordinates into an asynchronous action request using the nav2_msgs/action/NavigateToPose framework. It dynamically configures the navigation message header to reference the global coordinates mapping grid (map), stamps it with current simulation time markers, initializes the position coordinates, and forces a neutral forward-facing orientation vector $w = 1.0$.

- **Goal Evaluation and Lifecycle Management:** The request is sent to the central navigation action server via a non-blocking future callback mechanism (send_goal_async). The node tracks the future's state transitions: if the goal is rejected by Nav2 due to costmap blockages, a warning log is generated. If accepted, it shifts into a tracking state.
- **Success Handshaking:** Upon successful arrival at the goal location, the server returns a definitive completion result code (4, indicating a SUCCEEDED status). The client immediately matches this code to trigger a boolean confirmation payload, publishing it over the internal target_reached topic to activate the water pump node and launch the suppression cycle.

7. Conclusion

The hardware and electrical system of the Intelligent Mobile Robot integrate high-performance computing, efficient power management, and robust actuation into a cohesive platform. By combining local edge vision processing with autonomy, the robot achieves reliable fire detection and suppression while maintaining modularity and scalability. This architecture provides a strong foundation for future enhancements in navigation, perception, and industrial deployment.

References

1. Ren, X., Qu, K., Guo, J., Yin, H. and Huang, B., 2025. Research on accurate fire source localization and seconds-level autonomous fire extinguishing technology. *Scientific Reports*, *15*(1), p.17135.
2. Yuvaraj, R., Bhalerao, S.A., Murugesan, K., Vellaiyan, S. and Van Minh, N., 2025. Real-Time Fire Detection and Suppression System Using YOLO11n and Raspberry Pi for Thermal Safety Applications. *Case Studies in Thermal Engineering*, p.107159.
3. Raspberry Pi Foundation, *Raspberry Pi 4 Model B Product Brief*, Raspberry Pi Ltd., 2023.
4. Raspberry Pi Foundation, *Raspberry Pi 4 Model B Datasheet and Hardware Documentation*, Raspberry Pi Ltd., 2023.
5. Sony Semiconductor Solutions Corporation, *IMX708 CMOS Image Sensor Datasheet*, Sony Corp., 2022.
6. Raspberry Pi Foundation, *Camera Module 3 Technical Documentation*, Raspberry Pi Ltd., 2023.
7. TP-Link Technologies Co., Ltd., *Tapo C210 Pan/Tilt Home Security Wi-Fi Camera Datasheet*, TP-Link, 2022.
8. XLSEMI, *XL4015 5A Adjustable Step-Down DC-DC Converter Datasheet*, XLSEMI Microelectronics, 2020.